Nederpelt & Geuvers §1.3: Lambda-terms

William Sollers

Notes, draft 1

1 The set of λ-terms

Assumption 1.1 (Variables)**.** *Source.* N&G §1.3, p. 4 (PDF p. 33).
There is an infinite set $V = \{x, y, z, \dots\}$ of variables.

Remark (Open). The book does not say whether V is countable; the listing suggests it. **Verify** against the page before relying on it.

Definition 1.2 (The set Λ of all λ-terms)**.** *Source.* N&G Definition 1.3.2, p. 4 (PDF p. 33).

(1) (Variable) If $u \in V$, then $u \in \Lambda$.

(2) (Application) If $M, N \in \Lambda$, then $(MN) \in \Lambda$.

(3) (Abstraction) If $u \in V$ and $M \in \Lambda$, then $(\lambda u\,.\,M) \in \Lambda$.

The definition is *inductive*: (1), (2) and (3) are the only ways to construct elements of Λ (p. 4).

Motivation.

To write: From §1.2: functions as abstraction and application.

Quantified form.

To write: Test 2 (least set): the intersection form. Name the domain the outer quantifier ranges over. Then its negation: when is a string *not* in Λ?

2 Where finiteness comes from

Remark (Inductive definitions: the pattern). *Source.* Explanatory; general treatment in Enderton §1.4 (Induction and Recursion), pp. 34–44. Construction sequences first appear in Enderton §1.1.

The pattern. An inductive definition has three parts:

(i) a *base*: a set of starting objects;

(ii) *rules*: finitely many operations, each taking finitely many objects and producing a new one;

(iii) a *closure clause*: nothing else is in the set.

For Λ: the base is V; the rules are application (two inputs) and abstraction (a variable and one term); the closure clause is the sentence after Definition 1.3.2. The same pattern defines $\mathbb{N}$ (base 0, rule successor), Enderton's WFF (base sentence symbols, rules the connectives), derivations, typing derivations, and every Lean `inductive` type.

The rules recurse. Rules (2) and (3) are recursive because they refer to Λ itself: they take terms in and produce terms. There is no separate recursion clause; the self-reference is the recursion.

The process never stops. The rules apply again and again: (MN), $((MN)(MN))$, ... are all terms. Arranged in stages,

$$\Lambda_0 = V, \qquad \Lambda_{n+1} = \Lambda_n \cup \{(MN) : M, N \in \Lambda_n\} \cup \{(\lambda u.M) : u \in V,\ M \in \Lambda_n\}, \qquad \Lambda = \bigcup_n \Lambda_n.$$

There is no fixed bound on term size, but each individual term is finite.

Two separate facts, as in $\mathbb{N}$:

- *No largest element.* A term at stage n has a larger one at stage $n+1$.
- *No infinite element.* An endless expression such as $\lambda x.\lambda x.\lambda x.\cdots$ is not a member. This is not a limit on what can be chosen; the object is not in the set.

$\mathbb{N}$ has no largest number, and ∞ is not a number.

Where finiteness is enforced. The closure clause is informal ("the only ways to *construct*"); the finiteness hides in the word *construct*. It is made precise in one of two ways, which define the same set (Enderton §1.4):

Reading	Definition	Finiteness
Bottom-up	last entry of a finite construction sequence	in the definition
Top-down	intersection of all sets closed under (1)–(3)	a theorem

In the top-down reading three ingredients enforce finiteness: leastness, a base of finite objects, and *finitary* rules. The set of finite strings is closed under the rules, and the least closed set lies inside every closed set. A rule with infinitely many inputs would let infinite terms in.

Dual. Taking the *greatest* set consistent with the rules instead (a coinductive definition) would admit infinite terms. "Least" is exactly the choice that excludes them. Lean's `inductive` always means the least.

Inductive, recursive, induction.

Notion	Does	Example
Inductive definition	defines a set	Λ (Def. 1.3.2)
Recursive definition	defines a function on it, by cases on the rules	Sub (Def. 1.3.5)
Proof by induction	proves a property of every element, by cases	Lemma 1.3.6

Both of the last two rest on leastness: every element came from some rule. Recursion needs one more hypothesis, *free generation* (unique readability): each term has exactly one last construction step. In Lean, one object, the recursor, provides both recursion and induction.

Definition 2.1 (Construction sequence for Λ)**.** *Source.* Not in N&G; Enderton's pattern (§1.1, §1.4) applied to Λ.

Let $A = V \cup \{(,), \lambda, .\}$, with $\frown$ for concatenation of strings over A. A *construction sequence* is a finite sequence $\langle t_0, \ldots, t_n \rangle$ of strings over A such that for each $i \leq n$ at least one holds:

(1) $t_i \in V$;

(2) $t_i = (\frown t_j \frown t_k \frown)$ for some $j, k < i$;

(3) $t_i = (\frown \lambda \frown u \frown . \frown t_j \frown)$ for some $u \in V$ and $j < i$.

A string t is *constructible* if it is the last entry of some construction sequence; Λ_* is the set of constructible strings.

Quantified form.

$$t \in \Lambda_* \iff \exists n\, \exists \langle t_0, \ldots, t_n \rangle \Big[t_n = t \ \wedge\ \forall i \leq n \big(t_i \in V \\ \vee\ \exists j, k < i.\ t_i = (t_j t_k) \\ \vee\ \exists u \in V\, \exists j < i.\ t_i = (\lambda u.\, t_j) \big) \Big]$$

Finiteness is explicit twice: in $\exists n$, and in $j, k < i$.

Theorem 2.2 (Every λ-term is finite)**.** *Source. Explanatory; pattern from Enderton §1.4.*

Every element of Λ is a finite string over A.

Remark (Proof structure). Let F be the set of finite strings over A. Check F satisfies (1)–(3); then leastness gives $\Lambda \subseteq F$.

Proof.

To write: Your proof.

□

Remark (Gaps).

- N&G assume, without proof, that the top-down and bottom-up readings agree. Proof: Enderton §1.4.

  To write: Your proof, in the Enderton notes.

- N&G define Sub by recursion on Λ without stating unique readability. **Verify** whether it is stated anywhere in Ch. 1.

Remark (Lean).

To write: After Enderton §1.4: the general node (universe, base, operations; $C^* = C_*$; induction) proposed for `Aim/MetaLanguage`, with Λ, WFF and $\mathbb{N}$ as instances; then the bridge showing a Lean `inductive` gives the same set.